HEALTHPARTNERS | PERSONALIZATION | DEV DAYS FOLLOW-UP

Campaign Runner: getting started

One config table and one Lakeflow job for every campaign, your shared pipeline stages kept as one chain, and delivery to Oracle that never skips or duplicates a message.

1 row
to add a campaign; nothing is deployed

1 job
replaces one DAG per campaign

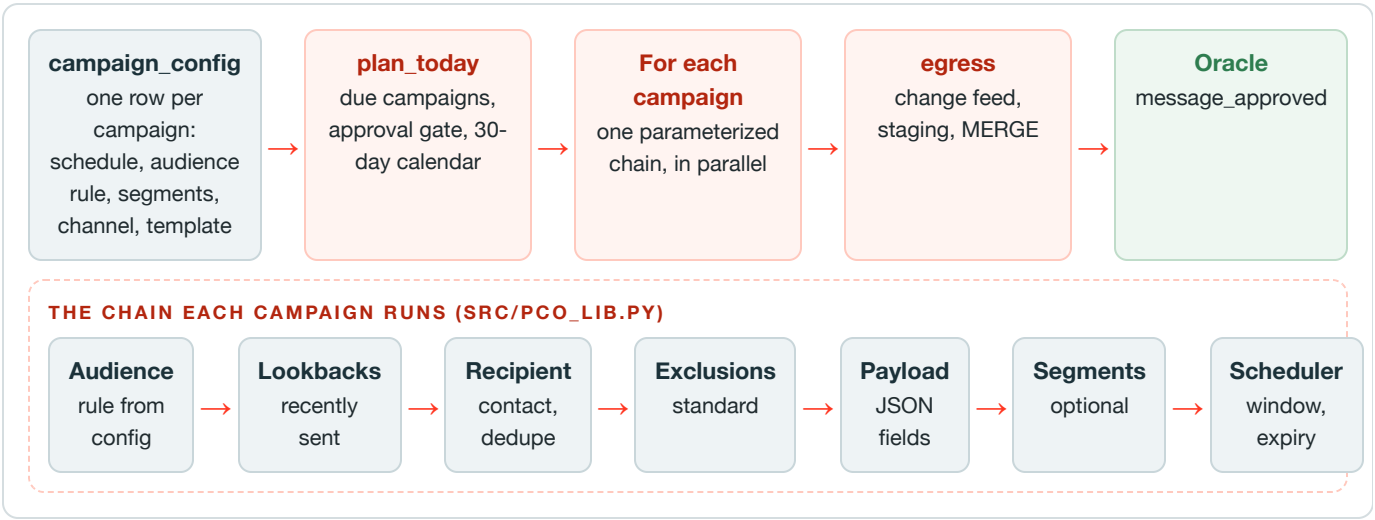
Exactly once
change data feed plus MERGE on message_guid

No patient rows
in the marketing dashboard and Genie Agent

What you asked for, and where it lives

Your objective	How the package addresses it
Consolidate 150+ campaign DAGs	Each campaign is a row in `campaign_config`. One **Campaign Runner** job runs every campaign that is due, in parallel. Cluster and runtime upgrades disappear with serverless; template changes are a new version row.
More efficient, direct Oracle egress	Delta **change data feed** instead of a `created_ts` watermark, staging plus **MERGE on message_guid** in the target, and delete-after-delivered cleanup. Three Oracle options, tested end to end on a Postgres target.
Marketing self-service	A **Campaign Operations** dashboard and Genie Agent over calendar, run history, delivery status and exceptions, with no person-level rows.
Keep what works	Your shared modules map one to one to functions in `pco_lib`, `message_approved` keeps its name, and Airflow can stay the scheduler with one DAG.

How it fits together



Your architecture, mapped to the pattern

Nothing is thrown away

Every stage of the current flow has a direct home. What changes is where the campaign-specific parts live: in rows instead of DAGs.

Today		In Campaign Runner
Airflow triggers each campaign on its own schedule	→	`schedule_cron` per row; the job's `plan_today` task picks what is due. Optional: one Airflow DAG triggers the job (`airflow/campaign_runner_dag.py`).
1 **Audience query** (campaign-specific)	→	`audience_rule`: a validated SQL predicate over your `audience_source` view, for example `is_birthday_month AND plan_type = 'Medicare'`
2 **Lookbacks**: remove recently sent	→	`lookbacks()` with `lookback_days` and scope (campaign, channel or any), reading `send_history`
2 **Target to recipient**: resolve contact, dedupe	→	`target_to_recipient()` over your `recipient_source` view; opt-in required per channel
2 **Exclusions**	→	`standard_exclusions()`: deceased, do-not-contact, `suppression_list`
2 **Write JSON payload**	→	`build_payload()`: the template's `payload_fields`, versioned in `templates`
2 **Segmentation** (campaign-specific)	→	`segmentation_rules` in config, for example `[{"segment": "65 and over", "rule": "age >= 65"}]`
2 **Message approved scheduler**	→	`schedule_messages()`: send window and expiration from config
3 **message_approved**	→	Same name. MERGE on a stable `message_guid`; unchanged rows are left alone; change data feed on
Daily job deletes rows **older than 7 days**	→	`maintenance`: deletes only **delivered** rows after `retention_days`
4 On-premises Python **incremental pull** on `created_ts`	→	`egress`: change data feed from the last delivered version, staging plus MERGE, then mark delivered

TIP

The campaign-specific code you maintain today becomes two columns: `audience_rule` and `segmentation_rules`. Everything else is one shared chain, changed once and applied to every campaign.

What's in the package

campaign-runner/

Path	What it is
`databricks.yml`	The bundle: variables and the dev, staging and prod targets. The only file you must edit.
`resources/`	The setup and daily jobs, the dashboard and the Genie Agent.
`src/pco_lib.py`	The shared pipeline stages, one function each, plus scheduling and rule validation.
`src/plan_today.py`, `run_campaign.py`	Pick due campaigns; run one campaign through the chain and MERGE into `message_approved`.
`src/egress.py`, `maintenance.py`, `run_summary.py`	Exactly-once delivery; delivery-aware cleanup; run report that fails loudly.
`src/00_sample_data.py`, `00_setup.py`	Synthetic sources and the two source views; tables, views and 11 sample campaigns.
`src/sql/`	Templates: add a campaign, change a template, your source views, Oracle DDL, delivery checks.
`airflow/`, `scripts/`, `docs/ORACLE_EGRESS.md`	The one-DAG Airflow option; dashboard and Genie generators; the egress options in detail.

Get running in dev

About 20 minutes

1. **Configure.** In `databricks.yml` under `targets.dev`, set the workspace host, your catalog and a SQL warehouse ID.
2. **Deploy.**

```
databricks auth login --host https://<your-dev-workspace>.azuredatabricks.net
databricks bundle deploy -t dev
```

3. **Set up.** Synthetic sources, tables, views and 11 sample campaigns:

```
databricks bundle run -t dev campaign_runner_setup
```

If the first deploy reported an error on the Genie Agent, deploy again now that its tables exist.

4. **Run.**

```
databricks bundle run -t dev campaign_runner
```

Open the run: the fan-out shows one run per due campaign, and `run_history` shows the count after each stage.

5. **Run it again.** Unchanged messages are left alone, so nothing new is written or resent.

> **YOU SHOULD SEE**
> In our test on synthetic data, eight campaigns produced 17,957 messages, with each run taking about 8 minutes. A rerun changed nothing. A new campaign row added only its own 1,731 messages, and a new template version updated only that campaign's 2,500 messages, each delivered once.

Add or change a campaign

Rows, not code

- **Add a campaign:** one insert (`src/sql/add_campaign_template.sql`). Write quotes inside a rule as `\'` or wrap the rule in double quotes.
- **Change a template:** add a version row. Changed messages for the current run go back to pending and are redelivered once, as updates.
- **Regulatory notices:** `requires_approval = true` holds the campaign as `AWAITING_APPROVAL` until someone sets `approved_by` .
- **Use your data:** define `audience_source` and `recipient_source` over your tables (`src/sql/audience_source_template.sql`). Campaign rules can use any column of `audience_source` .

Delivery to Oracle

docs/ORACLE_EGRESS.md

Today

A timestamp watermark can skip late or tied rows, and a 7-day age-based delete can remove messages Oracle never received. Neither raises an error.

With this package

Change data feed delivers every change exactly once by commit version; MERGE on `message_guid` makes retries harmless; cleanup deletes only delivered rows, and anything undelivered shows as `NOT_DELIVERED` .

A. Push from Databricks

Unity Catalog JDBC connection appends to an Oracle staging table; a separate step MERGEs on `message_guid` . Test this first. Template in `egress.py` , not executed.

B. Oracle pulls a share

Oracle Autonomous Database can read an OpenSharing share. Share a table without deletion vectors. Confirm support for other Oracle deployments.

C. Keep the script, governed

Run it as a job task on the change data feed and checkpoint, with delete-after-delivered cleanup. Smallest change.

IMPORTANT Answer the network question first: serverless compute must reach Oracle for option A. An on-premises or private Oracle needs private connectivity or a firewall allowlist for serverless.

Spark JDBC can append to or overwrite a table but cannot run MERGE, UPDATE or DELETE, which is why the MERGE is its own step. Measure rows per second, MERGE time, rerun behaviour and Oracle load in non-production before choosing.

Staging and prod

CI/CD

dev
Each developer's copy. Sample data, paused schedule.

→

staging
Deployed by CI on merge to main, run by a service principal.

→

prod
Deployed on approval. Your sources and Oracle; daily schedule on.

1. Create a service principal per environment with access to the catalog and serverless jobs.
2. Set the host, catalog, warehouse and egress variables for `staging` and `prod`.
3. In GitHub, add `DATABRICKS_HOST`, `DATABRICKS_CLIENT_ID` and `DATABRICKS_CLIENT_SECRET` to the `staging` and `prod` environments, with required reviewers on `prod`. The workflows skip themselves until `DATABRICKS_HOST` is set.

Marketing self-service

No patient rows

The dashboard and Genie Agent read only campaign-level views: `calendar`, `run_history`, `message_summary` and `run_exceptions`. Marketing can ask what goes out tomorrow, which regulatory notices are next week, what failed or was not delivered, and what is waiting for approval, without asking an engineer.

Exception state	Meaning
FAILED	A campaign run failed. The summary task also turns the job run red.
NOT_DELIVERED	Messages are still pending after their send window ended.
LATE	The send date passed with no run.
AWAITING_APPROVAL	A regulatory campaign is scheduled but not approved.

TIP
Pilot the Genie Agent with the five questions marketing asks your team most often, and turn them into example questions in `scripts/build_genie_resource.py`.

Our recommendations

From the session

1. A campaign is a row
Move each campaign's audience rule and segmentation into config; keep the shared stages as one chain.

2. Retire before you migrate
Inventory the DAGs; one-time campaigns that already ran can simply be retired.

3. Coexist with Airflow first
One DAG triggers the job; move scheduling later only if it pays off.

4. Fix delivery safety now
Change data feed, MERGE on `message_guid`, delete only what was delivered.

5. Measure the Oracle options
Test option A in non-production with your DBA; let the numbers choose.

6. Templates and the job in Git
Bundles plus GitHub Actions: one pull request per change.

7. Self-service for marketing
The Genie Agent and dashboard, piloted with real questions.

8. Know what it costs
Tag the job with serverless usage policies by owning team.

Suggested first 30 days

Start small

WEEK 1

Inventory the DAGs
Group by template, cadence and owner; mark one-time campaigns to retire.

WEEK 2

10-campaign pilot
Deploy the bundle in non-production; your source views; ten campaigns as rows.

WEEK 3

Oracle test
Non-production schema and driver; measure option A; confirm the network path.

WEEK 4

Marketing pilot
A few users on the Genie Agent and dashboard; alerts on.

Decisions to make

Decision	Owner	By when	Notes
The Oracle option and the non-production schema and driver for the test			
Network path from serverless to Oracle			
The pilot set: the first ten campaigns			
Scheduler: keep Airflow triggering one job, or schedule in Databricks			

Learn more

- Declarative Automation Bundles and Lakeflow Jobs (for each, Airflow)
- Change data feed and JDBC connections in Unity Catalog
- Serverless network security and Genie Agents

The sample data in this package is synthetic. Results on your data will differ.